

1. Работа с данными сотрудников

Список активных

Версия API: 1.0

Версия iiko: 4.0

GET <https://host:port/resto/api/employees>

Параметры запроса

Название	Значение	Версия	Описание
includeDeleted	true/false	с 5.0	Возвращать и действующих, и удаленных сотрудников
revisionFrom	число, номер ревизии	с 6.4	Номер ревизии, начиная с которой необходимо отфильтровать сущности. Не включающий саму ревизию, т.е. ревизия объекта > revisionFrom. По умолчанию (неревизионный запрос) revisionFrom = -1

Что в ответе

Список сотрудников. Все сотрудники (включая встроенные системные аккаунты), которые активны (не удалены)

Пример запроса

```
https://localhost:8080/resto/api/employees?key=284c5690-2b56-b1d6-0c81-e94b5034243d
```

Список по подразделению

Версия API: 1.0

Версия iiko: 4.0

GET <https://host:port/resto/api/employees/byDepartment/{departmentCode}>

Параметры запроса

Параметр	Описание
includeDeleted (с 5.0)	Возвращать и действующих, и удаленных сотрудников

Что в ответе

Список сотрудников указанного подразделения. Все сотрудники (включая встроенные системные аккаунты), которые активны (не удалены).

Для RMS идентично обычному списку, для Chain - только список сотрудников указанного подразделения.

Пример запроса

```
https://localhost:8080/resto/api/employees/byDepartment/1?key=284c5690-2b56-b1d6-0c81-e94b5034243d
```

Сотрудник по ID

Версия API: 1.0

Версия iiko: 4.0

GET <https://host:port/resto/api/employees/byId/{employeeUUID}>

Что в ответе

Сотрудник с указанным GUID.

Пример запроса

```
https://localhost:8080/resto/api/employees/byId/61b97cfb-6b4e-4668-9a38-c30190f7a109?key=180c8a55-efae-8183-0d6d-015a685f84f1
```

Сотрудник по коду

Версия API: 1.0

Версия iiko: 4.0

GET

https://host:port/resto/api/employees/byCode/{employeeCode}

Что в ответе

Сотрудник с указанным кодом.

Пример запроса

```
https://localhost:8080/resto/api/employees/byCode/2?key=a10b7fdc-9ae5-449f-c6fb-cb5a67e5b2e6
```

Поиск сотрудника

Версия API: 1.0

Версия iiko: 4.0

GET

https://host:port/resto/api/employees/search?firstName={regex}&middleName={regex}

Параметры запроса

Параметры	Описание
address cardNumber cellPhone client	Регулярное выражение По любому из текстовых или булевых полей в dto (см. список в 1 столбце)

code email employee firstName lastName login mainRoleCode middleName name note phone supplier	Параметры необязательные. Если отсутствуют, вернет всех активных
includeDeleted (с 5.0)	Возвращать и действующих, и удаленных сотрудников

Что в ответе

Сотрудник с указанными именем и/или отчеством.

Пример запроса

```
https://localhost:8080/resto/api/employees/search?key=de7c43fc-b4d7-cf45-51b4-c40cba21265f&firstName=n&middleName=m
```

Добавить или заменить сотрудника (по Id)

Версия API: 1.0

Версия iiko: 4.0

GET <https://host:port/resto/api/employees/byId/{UUID}>

Параметры запроса

Что в ответе

Если передан новый id, то будет создан новый сотрудник (код возврата 201 Created).

Если передан id существующего сотрудника, то произойдет полное замещение всех полей сотрудника (код возврата 200 OK). При этом если не указать какое-либо из необязательных полей, то значение этого поля сбросится.

Для обновления частичного набора полей используйте метод POST **/employees/byId/{employeeUUID}**

Пример запроса

```
https://localhost:8080/resto/api/employees/byId/4f390698-241d-6ab9-015e-a3d90baa0370
```

Copy Code

Код

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<employee>
  <code>6</code>
  <name>APIbyId</name>
  <login/>
  <mainRoleCode>CS1</mainRoleCode>
  <roleCodes>CS1</roleCodes>
  <phone>7979</phone>
  <cellPhone>00000</cellPhone>
  <firstName>Name</firstName>
  <lastName>Name</lastName>
  <birthday>2017-09-14T00:00:00+03:00</birthday>
  <email>email2@mail.ru</email>
  <address>address</address>
  <hireDate>2017-09-04T00:00:00+03:00</hireDate>
  <fireDate>2017-09-21T00:00:00+03:00</fireDate>
  <cardNumber/>
  <taxpayerIdNumber>111111111111111111</taxpayerIdNumber>
  <snils>455555555555555555</snils>
  <preferredDepartmentCode>1</preferredDepartmentCode>
  <departmentCodes>1</departmentCodes>
  <responsibilityDepartmentCodes>1</responsibilityDepartmentCodes>
  <deleted>false</deleted>
  <supplier>false</supplier>
  <employee>true</employee>
  <client>false</client>
  <publicExternalData>
    <entry>
      <key>keyPUT</key>
      <value>valuePUT</value>
    </entry>
  </publicExternalData>
</employee>
```

Пример результата вызова API

[Copy Code](#)

Код

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<employee>
  <id>4f390698-241d-6ab9-015e-a3d90baa0370</id>
  <code>6</code>
  <name>APIbyId</name>
  <login/>
  <mainRoleCode>CS1</mainRoleCode>
  <roleCodes>CS1</roleCodes>
  <phone>7979</phone>
  <cellPhone>00000</cellPhone>
  <firstName>Name</firstName>
  <lastName>Name</lastName>
  <birthday>2017-09-14T00:00:00+03:00</birthday>
  <email>email2@mail.ru</email>
  <address>address</address>
  <hireDate>2017-09-04T00:00:00+03:00</hireDate>
  <fireDate>2017-09-21T00:00:00+03:00</fireDate>
  <cardNumber/>
  <taxpayerIdNumber>111111111111111111</taxpayerIdNumber>
  <snils>455555555555555555</snils>
  <preferredDepartmentCode>1</preferredDepartmentCode>
  <departmentCodes>1</departmentCodes>
  <responsibilityDepartmentCodes>1</responsibilityDepartmentCodes>
  <deleted>false</deleted>
  <supplier>false</supplier>
  <employee>true</employee>
  <client>false</client>
  <publicExternalData>
    <entry>
      <key>keyPUT</key>
      <value>valuePUT</value>
    </entry>
  </publicExternalData>
</employee>
```

Добавить сотрудника (по коду)

PUT<https://host:port/resto/api/employees/byCode/{employeeCode}>

Что в ответе

Новый сотрудник (код возврата 201 Created).

Примечание: учитывается только код, переданный в теле PUT-запроса.

Пример запроса

```
https://localhost:8080/resto/api/employees/byCode/5
```

employeeCode - введенное значение будет отображаться в поле Табельный номер

Пример результата вызова API

[Copy Code](#)

Код

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<employee>
  <id>4f390698-241d-6ab9-015e-a3d90baa0371</id>
  <code>5</code>
  <name>API</name> //имя в системе
  <login/>
  <mainRoleCode>CS1</mainRoleCode>
  <roleCodes>CS1</roleCodes>
  <phone>78787878</phone>
  <cellPhone>56565</cellPhone>
  <firstName>firstName</firstName>
  <lastName>lastName</lastName>
  <birthday>2017-09-14T00:00:00+03:00</birthday>
  <email>email@mail.ru</email>
  <address>address</address>
  <hireDate>2017-09-04T00:00:00+03:00</hireDate>
  <fireDate>2017-09-21T00:00:00+03:00</fireDate>
  <cardNumber/>
  <taxpayerIdNumber>111111111111111111</taxpayerIdNumber>
  <snils>4555555555555555</snils>
  <preferredDepartmentCode>1</preferredDepartmentCode>
  <departmentCodes>1</departmentCodes>
  <responsibilityDepartmentCodes>1</responsibilityDepartmentCodes>
  <deleted>false</deleted>
  <supplier>false</supplier>
  <employee>true</employee>
  <client>false</client>
</employee>
```

Изменить/добавить сотрудника (по id)

Версия API: 1.0

Версия iiko: 4.0

POST <https://host:port/resto/api/employees/byId/{employeeUUID}>

Что в ответе

Если передан новый id, то будет создан новый сотрудник (код возврата 201 Created).

Если передан id существующего сотрудника, то произойдет изменение указанных полей (код возврата 200 OK). Поля не указанные в запросе останутся без изменений.

Для полного замещения всех полей используйте метод PUT **/employees/byId/{employeeUUID}**.

Список пар ключ-значение в поле publicExternalData задается через XML следующим образом:

```
<r><entry><key>key_POST1</key><value>value_POST1</value></entry><entry><key>key_POST2</key><value>value_POST2</value></entry></r>.
```

Корневой тег может быть любым, можно задавать `<r>` для краткости, главное чтобы содержимое соответствовало указанному формату. Сервер "знает", что нужно распарсить это поле в виде XML и положить в виде списка пар ключ-значение в справочник User.

Внутри `<entry>` ключ `<key>` задавать обязательно, т.е. ключ не может быть null, а вот значение `<value>` можно не задавать, поскольку оно может быть null, см. employee.xsd ниже.

Пример запроса

```
https://localhost:8080/resto/api/employees/byId/4f390698-241d-6ab9-015e-a3d90baa0370
```

[Copy Code](#)

Код

```
<!--Content-Type: application/x-www-form-urlencoded-->code=10&name=name&taxpayerIdNumber=000000000000&externalData=<r><entry><
```



```
key>key_POST1</key><value>value_POST1</value></entry><entry><key>key_POS  
T2</key><value>value_POST2</value></entry></r>
```

Пример результата вызова API

Copy Code

Код

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>  
<employee>  
  <id>4f390698-241d-6ab9-015e-a3d90baa0370</id>  
  <code>10</code>  
  <name>name</name>  
  <login/>  
  <mainRoleCode>CS1</mainRoleCode>  
  <roleCodes>CS1</roleCodes>  
  <phone>7979</phone>  
  <cellPhone>00000</cellPhone>  
  <firstName>Name</firstName>  
  <lastName>Name</lastName>  
  <birthday>2017-09-14T00:00:00+03:00</birthday>  
  <email>email2@mail.ru</email>  
  <address>address</address>  
  <hireDate>2017-09-04T00:00:00+03:00</hireDate>  
  <fireDate>2017-09-21T00:00:00+03:00</fireDate>  
  <cardNumber/>  
  <taxpayerIdNumber>000000000000</taxpayerIdNumber>  
  <snils>4555555555555555</snils>  
  <preferredDepartmentCode>1</preferredDepartmentCode>  
  <departmentCodes>1</departmentCodes>  
  <responsibilityDepartmentCodes>1</responsibilityDepartmentCodes>  
  <deleted>false</deleted>  
  <supplier>false</supplier>  
  <employee>true</employee>  
  <client>false</client>  
  <publicExternalData>  
    <entry>  
      <key>key_POST1</key>  
      <value>value_POST1</value>  
    </entry>  
    <entry>  
      <key>key_POST2</key>  
      <value>value_POST2</value>  
    </entry>  
  </publicExternalData>  
</employee>
```

Удалить сотрудника

Версия API: 1.0

Версия iiko: 4.0

DELETE

`https://host:port/resto/api/employees/byId/{employeeUUID}`

Что в ответе

Пустой ответ если сотрудник удален (или уже был удален).

Entity of class User not found by id (employeeUUID), если передан несуществующий guid.

Пример запроса

```
https://localhost:8080/resto/api/employees/byId/4f390698-241d-6ab9-015e-a3d90baa0370
```

Описание сущностей для представления в формате XML (XSD-схема)

[+] [Сотрудник](#)

[-] [Сотрудник](#)

[Copy Code](#)

XML

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<xs:schema version="1.0" xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="employee" type="employee"/>

  <xs:complexType name="employee">
    <xs:sequence>
      <xs:element name="id" type="xs:string" minOccurs="1"/>
      <!-- Табельный номер сотрудника. Пуст у системных учетных
```

```

записей. -->
    <xs:element name="code" type="xs:string" minOccurs="1"/>
    <xs:element name="name" type="xs:string" minOccurs="1"/>
    <!-- Логин для входа в бекофис -->
    <xs:element name="login" type="xs:string" minOccurs="0"/>
    <!-- Пароль для входа в бекофис. Доступен только на
изменение, не отображается. -->
    <xs:element name="password" type="xs:string" minOccurs="0"/>

    <!--
    Основная должность сотрудника. Входит в rolesIds.
    -->
    <xs:element name="mainRoleId" type="xs:string"
minOccurs="0"/>
    <xs:element name="rolesIds" type="xs:string" nillable="true"
minOccurs="0" maxOccurs="unbounded"/>

    <!--
    Основная должность сотрудника.
    Входит в roleCodes (кроме системных учетных записей, у них
не заполнено roleCodes).
    -->
    <xs:element name="mainRoleCode" type="xs:string"
minOccurs="0"/>
    <xs:element name="roleCodes" type="xs:string"
nillable="true" minOccurs="0" maxOccurs="unbounded"/>
    <!-- Справочная информация -->
    <xs:element name="phone" type="xs:string" minOccurs="0"/>
    <xs:element name="cellPhone" type="xs:string"
minOccurs="0"/>
    <xs:element name="firstName" type="xs:string"
minOccurs="0"/>
    <xs:element name="middleName" type="xs:string"
minOccurs="0"/>
    <xs:element name="lastName" type="xs:string" minOccurs="0"/>
    <xs:element name="birthday" type="xs:dateTime"
minOccurs="0"/>
    <xs:element name="email" type="xs:string" minOccurs="0"/>
    <xs:element name="address" type="xs:string" minOccurs="0"/>
    <xs:element name="hireDate" type="xs:string" minOccurs="0"/>
    <xs:element name="hireDocumentNumber" type="xs:string"
minOccurs="0"/>
    <!--
    Дата увольнения.
    @since 5.4
    -->
    <xs:element name="fireDate" type="xs:date" nillable="true"
minOccurs="0"/>
    <xs:element name="note" type="xs:string" minOccurs="0"/>
    <!-- Slip карты сотрудника -->
    <xs:element name="cardNumber" type="xs:string"
minOccurs="0"/>
    <!-- Pin-код для входа в iikoFront. Доступен только на

```

```

изменение, не отображается. -->
    <xs:element name="pinCode" type="xs:string" minOccurs="0"/>
    <!-- ИНН -->
    <xs:element name="taxpayerIdNumber" type="xs:string"
minOccurs="0"/>
    <!--
    СНИЛС.
    @since 5.4
    -->
    <xs:element name="snils" type="xs:string" nillable="true"
minOccurs="0"/>
    <!--
    Global Location Number для поставщиков
    @since 6.0
    -->
    <xs:element name="gln" type="xs:string" minOccurs="0"/>
    <xs:element name="activationDate" type="xs:dateTime"
minOccurs="0"/>
    <xs:element name="deactivationDate" type="xs:dateTime"
minOccurs="0"/>
    <!--
    Подразделение (одно из departmentCodes), в котором
    сотрудники назначаются смены в первую очередь.
    @since 5.0
    -->
    <xs:element name="preferredDepartmentCode" type="xs:string"
minOccurs="0"/>
    <!--
    Назначенные подразделения.
    Если null - сотруднику назначены все подразделения
    (существующие и будущие).
    -->
    <xs:element name="departmentCodes" type="xs:string"
nillable="true" minOccurs="0"
maxOccurs="unbounded"/>
    <!--
    Подразделения, в которых сотрудник является ответственным.
    Если null - сотруднику является ответственным во всех
    существующих и будущих подразделениях.
    -->
    <xs:element name="responsibilityDepartmentCodes"
type="xs:string"
nillable="true" minOccurs="0"
maxOccurs="unbounded"/>

    <xs:element name="deleted" type="xs:string" minOccurs="0"/>
    <xs:element name="supplier" type="xs:string" minOccurs="0"/>
    <xs:element name="employee" type="xs:string" minOccurs="0"/>
    <xs:element name="client" type="xs:string" minOccurs="0"/>

    <!--
    Произвольные данные в формате строковый ключ -> строковое
    значение,

```

семантика которых определяется внешней системой.
Сервер никак не интерпретирует эти данные.
-->

```
<xs:element name="publicExternalData">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="entry" minOccurs="0"
maxOccurs="unbounded">
        <xs:complexType>
          <xs:sequence>
            <xs:element name="key"
type="xs:string"/>
            <xs:element name="value"
type="xs:string" minOccurs="0"/>
          </xs:sequence>
        </xs:complexType>
      </xs:element>
    </xs:sequence>
  </xs:complexType>
</xs:element>
</xs:sequence>
</xs:complexType>
</xs:schema>
```